



Dziedziczenie - absolutne podstawy

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

- Klasa bazowa - klasa pochodna
 - lepiej jeżeli przypuszczamy, że będą też inne klasy pochodne
- Klasa bazowa - ogólna, zawiera części wspólne
- Klasa(y) pochodna(e) - szczegółowe,
 - rozszerzają funkcjonalność klasy bazowej,
 - doprecyzowują zachowania.



Dziedziczenie - po co?

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

- Żeby się nie powtarzać (czyli znowu DRY)
 - jeżeli mamy podobny kod
- operować na ogólniejszych pojęciach
 - ale pozwalać też na precyzyjną implementację szczegółów



Dziedziczenie - co daje?

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

- Dostęp do wszystkich elementów klasy bazowej
- implementacja wspólnych rzeczy w klasie bazowej



Dziedziczenie - podstawy

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

- Patrząc precyzyjnie każda klasa w pythonie dziedziczy po jakiejś klasie:
 - jeżeli wskażemy klasę wprost, to po wskazanej klasie
 - jeżeli nie wskażemy klasy, to po klasie object, możemy również wprost napisać, że dziedziczymy z object
 - W Pythonie 2.1 $< x < 3.0$ dawało to istotne różnice
 - W Pythonie > 3.0 nie ma to znaczenia
 - mimo to niektórzy zalecają podawanie wprost dziedziczenia po object
 - ale to średnio popularna konwencja (prawdopodobnie dla kompatybilności wstecznej).

Dziedziczenie - jak to w kodzie zapisać?

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

Dziedziczenie oznaczamy przez podanie po nazwie klasy pochodnej nazwy klasy bazowej w nawiasach

```
class Pochodna( Bazowa ) :
```

```
    pass
```

```
class Derived( Base ) :
```

```
    pass
```

```
class Reka( Organ ) :
```

```
    pass
```

W powyższych przykładach klasa Pochodna dziedziczy po klasie Bazowa, klasa Derived po klasie Base, a klasa Reka po klasie Organ.



Dziedziczenie - podstawy

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

Często w odniesieniu do relacji dziedziczenia klas:

- klasę bazową nazywa się po angielsku superclass, parent, albo base class,
- klasę pochodną nazywa się po angielsku subclass albo derived.



Dziedziczenie - przykład

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

Założmy, że mamy system do zarządzania uczelnią/szkołą i w tym systemie mamy zaplanować klasy opisujące użytkowników, ale wiemy, że będą tam różne funkcjonalności dla studentów/uczniów i nauczycieli. Wiemy również, że użytkownicy mają cechy wspólne: imię, nazwisko, login (oczywiście w rzeczywistym systemie będzie tych cech więcej, ale dla dobra przykładu pozostanemy przy 3).

```
class Osoba:
    def __init__(self, imie, nazwisko, login):
        self.imie = imie
        self.nazwisko = nazwisko
        self.login = login
class Nauczyciel(Osoba):
    pass
class Student(Osoba):
    pass
```

Dziedziczenie - przykład

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

Zacznijmy od tego, czego nie można zrobić:

```
osobaB = Nauczyciel()
TypeError: __init__() missing 3 required positional
arguments: 'imie', 'nazwisko', and 'login'
```

Domyślalmy się, że dla pozostałych 2 klas będzie podobnie:

```
osobaA = Osoba()
TypeError: __init__() missing 3 required positional
arguments: 'imie', 'nazwisko', and 'login'

osobaC = Student()
TypeError: __init__() missing 3 required positional
arguments: 'imie', 'nazwisko', and 'login'
```

Dziedziczenie - przykład

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

Jeżeli nie chcemy dokładać żadnych swoich atrybutów, to możemy wywołać `__init__` z klasy bazowej:

```
osobaB = Nauczyciel("Andrzej", "Andrzej", "andrzej")  
>>> print(osobaB.imie)  
Andrzej
```

Dziedziczenie - przykład

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

O ile po przyjrzeniu się klasie `Osoba` jesteśmy w stanie przypuszczać, że będzie dokładnie jak na poprzednim slajdzie, o tyle zachowanie dwóch pozostałych klas wymaga wyjaśnienia:

- klasy pochodne dziedziczą atrybuty i implementacje metod z klas bazowych,
- klasy pochodne mogą nadpisywać implementacje metod z klas bazowych.
 - w powyższym przykładzie nie miało to miejsca więc obowiązuje `__init__` z klasy bazowej.



Dziedziczenie - przykład

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

Nadpiszemy więc metodę `__init__`, na razie w klasie `Nauczyciel`

```
class Nauczyciel(Osoba):  
    def __init__(self):  
        pass
```

i oczywiście teraz mamy sukces...

```
>>> osobaB=Nauczyciel()  
>>> osobaB  
<__main__.Nauczyciel object at 0x02B1EC90>
```

Dziedziczenie - przykład

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

...tylko czy pełny?

```
>>> osobaB=Nauczyciel()
>>> osobaB
<__main__.Nauczyciel object at 0x02B1EC90>
print(osobaB.imie)

>>> print(osobaB.imie)
Traceback (most recent call last):
  File "<pyshell#11>", line 1, in <module>
    print(osobaB.imie)
AttributeError: 'Nauczyciel' object has no attribute
'imie'
```

Więc wstrzymajmy się jeszcze chwilę z dopisywaniem `__init__` do klasy Student.

- Powyższe podejście jest **bliższe** podejściu do implementacji tego typu rozwiązań o dumnie brzmiącej nazwie top-down,
- w przeciwieństwie do łatwiejszego (zależy dla kogo i kiedy) podejścia bottom-up,
- nazwy chyba dlatego zupełnie inaczej brzmią, żeby się dobrze rozróżniało te podejścia.

Dygresja do dygresji: 1, 2, many jako metoda znajdowania podobieństw.

Dziedziczenie - przykład

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

Wróćmy jednak do naszych klas i metody `__init__`, spróbujmy ją trochę lepiej zaimplementować:

```
class Nauczyciel(Osoba):  
    def __init__(self):  
        self.imie = "ukryte"  
        self.nazwisko = "ukryte"  
        self.login = "ukryte"
```

```
>>> osobaB = Nauczyciel()  
>>> print(osobaB.imie)
```

```
ukryte
```

Dziedziczenie - przykład

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

Czyli da się lepiej:

```
class Nauczyciel(Osoba):
    def __init__(self, imie, nazwisko, login):
        self.imie = imie
        self.nazwisko = nazwisko
        self.login = login

>>> osobaB = Nauczyciel("Andrzej", "Andrzej", "andrzej")
>>> print(osobaB.imie)
Andrzej
```

Dziedziczenie - przykład

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

Ale powyższe dalej jest powtarzaniem kodu (łamamy DRY).
Więc:

```
class Nauczyciel(Osoba):  
    def __init__(self, imie, nazwisko, login):  
        super().__init__(imie, nazwisko, login)
```

Funkcja `super` służy do pobrania klasy bazowej i np. wywołania z niej konkretnej metody (w tym wypadku `__init__`).

Która z metod zostanie wywołana? Co się stanie?:

```
class Osoba:
    def __init__(self, imie, nazwisko, login):
        self.imie = imie
        self.nazwisko = nazwisko
        self.login = login

class Nauczyciel(Osoba):
    def __init__(self, imie, nazwisko):
        super().__init__(imie, nazwisko)

osobaB = Nauczyciel("Andrzej", "Andrzej", "andrzej")
osobaB = Nauczyciel("Andrzej", "Andrzej")
```

A teraz co się stanie?:

```
class Osoba:
    def __init__(self, imie, nazwisko, login):
        self.imie = imie
        self.nazwisko = nazwisko
        self.login = login

class Nauczyciel(Osoba):
    def __init__(self, imie, nazwisko):
        super().__init__(imie, nazwisko, "brak")

osobaB = Nauczyciel("Andrzej", "Andrzej", "andrzej")
osobaB = Nauczyciel("Andrzej", "Andrzej")
```



Dziedziczenie - podsumowanie

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

- Dziedziczenie atrybutów i metod działa dla wszystkich metod
- nadpisywać i wywoływać przez super można wszystkie metody nie tylko specjalne.

Dziedziczenie - co ze Studentem?

Programowanie
obiektywne

Dziedziczenie
- podstawy
Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

Moja propozycja, klasy Nauczyciel ani Osoba nie mają numeru indeksu, a tu jest:

```
class Student(Osoba):  
    def __init__(self, imie, nazwisko, login,  
                  nr_indeksu=0):  
        super().__init__(imie, nazwisko, login)  
        self.nr_indeksu=nr_indeksu
```

To przykład na to, jak klasy pochodne dokładają funkcjonalność do klas bazowych. Obsługa tego nowego pola powinna się znajdować tylko w klasie **Student**



Dziedziczenie z typów wbudowanych

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

- Możliwe jest w Pythonie dziedziczenie z typów wbudowanych (jak np. lista, słownik),
- tak naprawdę potrzebujemy nazwy typu wbudowanego (np. list, dict),
- i pomysłu na funkcję lub zbiór funkcji rozszerzających.

Dziedziczenie z typów wbudowanych - przykład

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

Dodajmy do listy znajdowanie średniej:

```
class BetterList(list):
    def find_average(self):
        result = sum(self)
        result = result / len(self)
        return result
```

```
x=BetterList()
x.append(1)
x.append(3)
x.append(2)
```

```
>>>print(x.find_average())
2.0
```

Dziedziczenie z typów wbudowanych - przykład

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

Można lepiej:

```
class BetterList(list):

    def __init__(self, *args):
        list.__init__(self, *args)

    def find_average(self):
        result = sum(self)
        result = result / len(self)
        return result
```

```
x=BetterList([1, 3, 5])
```

```
>>>print(x.find_average())
```

```
3.0
```



*args, **kwargs - co to?

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

Argumenty funkcji `*args` i `**kwargs` służą do przekazywania zmiennej liczby parametrów do funkcji:

- `*args` - do przekazania dowolnej liczby nie nazwanych parametrów do funkcji,
- `**kwargs` - do przekazania dowolnej liczby nazwanych parametrów (w postaci słownika).
- w przypadku kiedy chcemy skorzystać ze wszystkich rodzajów parametrów, funkcja musi mieć taki nagłówek:
`foo(lista_normalnych_parametrów, *args, **kwargs)`

*args - przykład

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

```
def foo(arg1, arg2, *argv):
    print("normalne argumenty:", arg1)
    for arg in argv:
        print("parametr *argv:", arg)
    print("normalne argumenty:", arg2)
```

```
>>>foo('Zdanie:', 'Ala', 'ma', 'komara', 1, 5)
normalne argumenty: Zdanie:
parametr *argv : ma
parametr *argv : komara
parametr *argv : 1
parametr *argv : 5
normalne argumenty: Ala
```

Dziedziczenie z typów wbudowanych - przykład

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

Teraz ten przykład powinien być lepiej zrozumiały:

```
class BetterList(list):

    def __init__(self, *args):
        list.__init__(self, *args)

    def find_average(self):
        result = sum(self)
        result = result / len(self)
        return result
```

```
x=BetterList([1, 3, 5])
```

```
>>>print(x.find_average())
3.0
```

**kwargs

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

```
def foo(**kwargs):
    if kwargs is not None:
        for key, value in kwargs.items():
            print ("klucz: %s = wartosc: %s" % (key,
                value))
```

```
>>>foo(kolor="niebieski", wysokosc=100, rodzaj="
    normalny")
klucz: rodzaj = wartosc: normalny
klucz: wysokosc = wartosc: 100
klucz: kolor = wartosc: niebieski
```

Alternatywne wywołanie:

```
kwargs = {"kolor": "niebieski", "wysokosc"
:100, "rodzaj": "normalny"}
foo(**kwargs)
```

Dziedziczenie wielobazowe, wielokrotne

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

- Dziedziczenie wielobazowe (ang. multiple inheritance) polega na dziedziczeniu z więcej niż jednej klasy
- trzeba przy tym uważać, ale wolno to robić
- z praktycznego punktu widzenia jest sensowne wtedy gdy tylko jedna lub żadna klasa nie jest konkretna,
- konkretna to taka, która posiada jakieś obiekty, w przykładzie z osobami, klasy Nauczyciel i Student były konkretne a Osoba nie, w ofercie sklepu mamy Przedmiot (klasa bazowa - ale nie konkretna) i Krzeslo oraz Stolik (klasy pochodne - konkretne).



Dziedziczenie wielobazowe - zapis

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

Dziedziczenie wielobazowe zapisujemy poprzez podanie kolejnych klas bazowych po przecinku wewnątrz nawiasów:

```
class Something(Czlowiek, Samochod):  
    pass  
  
class EmailableCustomer(Customer, MailInfo):  
    pass  
  
class PrintableShape(Shape, Printable):  
    pass  
  
class TowarMagazynowy(Przedmiot,  
    GospodarkaMagazynowa, OfertaKatalogowa):  
    pass
```

Dziedziczenie wielobazowe - uwaga

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

Dziedziczenie wielobazowe może doprowadzić do "problemu diamentu", kiedy 2 klasy dziedziczą po jednej, a następnie po nich wielobazowo dziedziczy jedna klasa:

```
class Base :  
    pass  
class LeftBase ( Base ) :  
    pass  
class RightBase ( Base ) :  
    pass  
class Derived ( LeftBase , RightBase ) :  
    pass
```

W uproszczeniu (bo temat jest zaawansowany):

- raczej nie powinno się tak robić,
- zawsze używać `super` a nie `LeftBase` albo `RightBase`.

Dziedziczenie - polimorfizm

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polimorfizm

Polimorfizm
Duck typing

- Polimorfizm - "wielopostaciowość"
- istnieje klasa bazowa i kilka jej klas pochodnych, które mają różnice w zachowaniach
 - polegające na różnej implementacji tych samych metod
- Obiekty zachowują się zgodnie z tym co zdefiniowano w klasach pochodnej,
- natomiast część operacji (jak np. inicjalizacja) może być zdefiniowana w klasie bazowej,
- Polimorfizm w Pythonie jest dość naturalny, poza dziedziczeniem nie ma tu jakichś specjalnych konstrukcji (w skrajnym przypadku nie potrzeba nawet dziedziczenia - duck typing).

Polymorfizm - przykład

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

```
class Obrazek:
    def __init__(self, filename):
        self.filename = filename
class ObrazekJpeg(Obrazek):
    def obejrzyj(self):
        print("ogladasz_obrazek_jpg:" + self.
              filename)
class ObrazekPng(Obrazek):
    def obejrzyj(self):
        print("ogladasz_obrazek_png:" + self.
              filename)
```

Polymorfizm - przykład

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

```
obrazki = [ObrazekJpeg("Pies.jpg"), ObrazekPng("logo
.png"), ObrazekJpeg("wakacje.jpg")]
>>>obrazki[0].obejrzyj()
ogladasz obrazek jpg: Pies.jpg
>>>obrazki[1].obejrzyj()
ogladasz obrazek png: logo.png
>>>obrazki[2].obejrzyj()
ogladasz obrazek jpg: wakacje.jpg
```

Polymorfizm - przykład

Programowanie
obiektowe

Dziedziczenie
- podstawy
Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

Można (i to często się robi) iterować po tak stworzonych obiektach.

```
obrazki = [ObrazekJpeg("Pies.jpg"), ObrazekPng("logo  
.png"), ObrazekJpeg("wakacje.jpg")]
```

```
>>>for obrazek in obrazki:  
    obrazek.obejrzyj()
```

```
ogladasz obrazek jpg: Pies.jpg  
ogladasz obrazek png: logo.png  
ogladasz obrazek jpg: wakacje.jpg
```



Dziedziczenie - duck typing

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
- args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

- jeśli chodzi jak kaczka i kwacze jak kaczka, to musi być kaczka



Dziedziczenie - duck typing

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

- Jeśli chodzi jak kaczka i kwacze jak kaczka, to musi być ...

Dziedziczenie - duck typing

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

- ... kaczką, albo ...



Dziedziczenie - duck typing

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

- ... kaczką, po prostu



Dziedziczenie - duck typing

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm

Duck typing

Duck typing daje pewną swobodę, jest też jednak obiektem pewnej krytyki. Jeżeli w poprzednim przykładzie dołożymy klasę:

```
class ObrazekNef():  
    def __init__(self, filename):  
        self.filename = filename  
    def obejrzyj(self):  
        print("ogladasz _obrazek_nef:_ " + self.  
              filename)
```

Dziedziczenie - duck typing

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

A następnie dodamy ją do listy obrazków (czyli kaczek):

```
obrazki = [ObrazekJpeg("Pies.jpg"), ObrazekPng("logo  
.png"), ObrazekJpeg("wakacje.jpg"), ObrazekNef("dozmiany.nef")]  
>>>for obrazek in obrazki:  
    obrazek.obejrzyj()  
ogladasz obrazek jpg: Pies.jpg  
ogladasz obrazek png: logo.png  
ogladasz obrazek jpg: wakacje.jpg  
ogladasz obrazek nef: dozmiany.nef
```



Dziedziczenie - duck typing

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

- Problemem jest brak możliwości określenia zawczasu, że klasa implementuje odpowiedni protokół.
 - czyli, że ma listę odpowiednich metod.
- Żeby to rozwiązać wprowadza się bazowe klasy abstrakcyjne (ABC - Abstract base classes).



Dziedziczenie - użycie ABC

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

```
class MojKontener:
    def __contains__(self, x):
        if not isinstance(x, int) or not x%2:
            return False
        return True

from collections import Container
odd = MojKontener()
print(isinstance(odd, Container))

print(1 in odd)
print(2 in odd)
print(3 in odd)
print(4 in odd)
```

```
True
True
False
True
False
```

```
import abc

class Player(metaclass=abc.ABCMeta):
    @abc.abstractmethod
    def play(self):
        pass

    @abc.abstractproperty
    def ext(self):
        pass

    @classmethod
    def __subclasshook__(self, C):
        if self is Player:
            attrs = set(dir(C))
            if set(self.__abstractmethods__) <= attrs:
                return True
            return NotImplemented
```



Dziedziczenie - własna ABC

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

```
class Wav( Player ):
    pass

#x=Wav()

class Ogg( Player ):
    ext='.ogg'
    def play( self ):
        pass

x=Ogg()
```

O co chodzi z @?

Programowanie
obiektywne

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych
args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm
Polymorfizm
Duck typing

W Pythonie za pomocą znaku @ oznaczamy dekoratory (ang. decorators), w innych językach nazywane atrybutami lub adnotacjami. Służą one jako metainformacje o funkcjach, klasach i metodach (czyli może z nich np. skorzystać interpreter Pythona lub inne narzędzie zewnętrzne. Ciekawy przykład:

```
def logowane(funkcja):
    def logowanie(*args, **kwargs):
        print("Podane argumenty: %s, %s" % (args,
            kwargs))
        return funkcja(*args, **kwargs)
    return logowanie
```

@logowane

```
def foo(x, y=1):
    return x * y
foo(3, 4)
foo(5, 6)
```



O co chodzi z @?

Programowanie
obiektowe

Dziedziczenie
- podstawy

Podstawy
przykład

Dziedziczenie
z typów
wbudowanych

args, kwargs

Dziedziczenie
wielobazowe

Polymorfizm

Polymorfizm
Duck typing

```
Podane argumenty : (3, 4), {}  
Podane argumenty : (5, 6), {}
```

Lepsze zobrazowanie:

```
print(foo(3, 4))  
print(foo(5))
```

```
Podane argumenty : (3, 4), {}  
12  
Podane argumenty : (5,), {}  
5
```